

Private inference as infrastructure

A systems architecture for running language models across independent providers without revealing prompts or activations

PLLM project

September 2026

PLLM turns a language-model request into a structured computation that can be split across separately operated services. Its current implementation targets public-weight transformer inference with an offline Preparation service, an online Inference service, and a client that retains plain-text, model state, and output decoding.

The project is open source, implemented as a Python package with a Rust native core, and designed to make protocol boundaries inspectable rather than hidden behind a conventional model endpoint.

The problem

Ordinary hosted inference asks users to trust a single provider with prompts, activations, and generation state. Encryption in transit protects the network path, but the server still receives the request in plaintext.

Private inference changes that trust boundary. The client must remain the only place where the request and model state are assembled, while remote providers perform useful model work on protected values.

PLLM focuses on the systems consequences of that requirement:

- model execution must be decomposed into explicit semantic and numeric operations;
- protocol roles need distinct identities, credentials, and state;
- offline work must be bound to the exact model and online computation;
- one-time material must be consumed or burned on every terminal path; and
- measurements must distinguish implemented paths from research proposals.

Current architecture

The current public-weight protocol has three roles.

For a public matrix W , client activation x , input mask r , and output mask s , Preparation computes $Wr - s$ before chat. During online execution, Inference receives $x - r$ and returns $W(x - r) + (Wr - s) = Wx - s$. The client adds s and center-decodes the result.

This moves heavy mask-correlated matrix work out of the token loop. The online path contacts only Inference.

Preparation never receives the online activation, and Inference never receives the preparation seed.

The client evaluates token lookup and the output head locally for public bundles. Transformer-body matrices stay remote. Prefill batches rows by stage; decode uses persistent authenticated WebSockets. Native Rust kernels select bounded 16-, 24-, or 32-bit rings from exact output bounds.

Security boundary

The current protocol assumes that Preparation and Inference follow the computation and do not collude. Preparation must erase expanded masks. Inference must enforce one-use inventory consumption. A single operator controlling both services can recombine protocol information and defeat the intended separation.

This is not an actively malicious security claim. The simulator, authenticated transports, and replay controls test implementation behavior; they do not prove security against arbitrary deviation. Self-hosting Preparation keeps its trust inside the client boundary, but it does not create independent operators.

PLLM also contains shared-transformer and garbling research components. They remain experimental. The preferred complete execution profile stays blocked until its operator coverage, truncation, quality, and security requirements are met.

Client	Preparation	Inference
Owns prompts, tokenization, nonlinear state, masks, scales, and decoding	Holds the public transformer body and computes offline masked corrections	Holds the same public body and evaluates masked online tensors
Sends fresh root seeds before a request	Expands each seed into one-use masks and uploads $W \mathbf{r} - \mathbf{s}$	Stores corrections in a sealed inventory
Sends only tickets and $\mathbf{x} - \mathbf{r}$ online	Is idle while tokens are generated	Returns $W \mathbf{x} - \mathbf{s}$; the client adds $\mathbf{s}$

What is implemented

The repository currently provides:

- semantic lowering for supported Qwen, Gemma, and Phi-family configurations;
- immutable plan and research-component APIs;
- native bounded matrix kernels and scalar reference paths;
- one-use prepared correction inventories;
- loopback benchmark and telemetry tooling;
- a local Responses API and Chat Completions API gateway; and
- reproducible paper, evidence, and documentation builds.

These are separate support axes. Lowering a complete semantic plan does not imply that the compiler can execute it. A transport benchmark does not establish output quality. A loopback deployment does not establish provider independence.

Evidence

The retained current-runtime study contains nine warm Qwen2.5-0.5B-Instruct runs on one CPU loopback host. It records exact workload dimensions, protocol byte counts, preparation state, latency distributions, and zero plaintext prompt or token telemetry bytes.

The cohort demonstrates repeatable execution of the implemented three-role transport. It does not establish WAN or GPU performance, energy use, operating price, concurrency, model quality, malicious security, or production non-collusion.

Every public claim should therefore remain bound to an immutable model body, protocol profile, source revision, workload, and evidence record.

Why the compiler matters

Privacy methods are not interchangeable wrappers around a model. Each method changes supported operators, numeric representation, communication, persistent state, and trust assumptions.

PLLM lowers model-specific configurations into a neutral decoder IR. Research transformations operate on semantic components rather than parsing adapter-specific names. Compilation then resolves numeric, protected, and placement graphs into an immutable plan. Unsupported coverage fails closed instead of silently moving private regions to plaintext execution.

This separation lets researchers compare methods against the same model semantics and makes missing coverage visible before deployment.

Direction

PLLM’s near-term objective is a complete, reproducible Qwen execution profile with explicit privacy and quality bounds. Longer-term work includes exact secure truncation, protected nonlinear execution, shared KV state, compact token selection, multi-host evaluation, and deployment evidence.

The project does not assume that one protocol will dominate every layer. Its architecture is built for composition: model adapters describe meaning, components describe transformations, compilers enforce coverage, and evidence records what actually ran.

Reproduce and inspect

The source repository contains the runtime, compiler, paper sources, benchmark records, and documentation. Start with the architecture, privacy boundaries, and reproduction guide.

Download the technical paper, current evidence record, or source archives from the research site.